MyCodeSchool

Testare cu JUnit și Mockito

Alex · teoria: forma unui test, mock-uri, @Mock, @InjectMocks, ArgumentCaptor.

Pornire

```
mvn test
```

Rulează toate clasele din `test/`. La final trebuie să vezi `Tests run: N, Failures: 0, Errors: 0` și `BUILD SUCCESS`. În IntelliJ: click dreapta pe `test/` → Run 'All Tests'.

Aplicația e în `src/`, testele în `test/`, iar `pom.xml` are deja JUnit 5 și Mockito. O clasă de test stă în același pachet ca clasa testată: `ProductTest` în `app.products.model`, lângă `Product`.

Forma unui test: Arrange, Act, Assert

```
@Test
void toStringReturnsTheSameLine() {
    // Arrange
    Product product = new Product("1,Laptop,3000,2000,IT,5");

    // Act
    String line = product.toString();

    // Assert
    assertEquals("1,Laptop,3000,2000,IT,5", line);
}
```

- **Arrange:** pregătești obiectele și datele.
- **Act:** un singur apel, metoda pe care o testezi.
- **Assert:** verifici rezultatul.

Numele spune ce se întâmplă și când: `addProductThrowsWhenNameAlreadyExists`. Când pică, din nume știi ce s-a stricat.

JUnit: verificări

Metodă	Trece dacă
`assertEquals(asteptat, actual)`	cele două sunt egale; primul argument e ce aștepți
`assertTrue(x)` / `assertFalse(x)`	`x` e `true` / `false`
`assertNull(x)` / `assertNotNull(x)`	`x` e / nu e `null`
`assertThrows(Tip.class, () -> apel)`	apelul aruncă o excepție de tipul dat

`assertThrows` primește apelul ca lambda: dacă l-ai scrie direct, excepția ar opri testul înainte să ajungă la verificare.

Ce e un mock

Un mock e un obiect fals de un anumit tip, creat de Mockito. Nu are logică:

- întoarce valoarea implicită (`null`, `0`, `false`, `Optional.empty()`, listă goală) până îi spui altceva;
- ține minte fiecare apel primit, cu argumentele lui.

De ce? Ca să testezi **o singură clasă**. Dacă `Cart` lucrează cu produse adevărate și testul pică, nu știi dacă greșeala e în `Cart` sau în `Product`. Cu mock-uri, greșeala e sigur în `Cart`.

Mockito	Ce face
`when(mock.metoda(arg)).thenReturn(valoare)`	fixează ce întoarce mock-ul
`when(mock.metoda(arg)).thenThrow(new X())`	mock-ul aruncă excepția
`verify(mock).metoda(arg)`	apelul a avut loc exact o dată, cu acel argument
`verify(mock, never()).metoda(any())`	apelul n-a avut loc deloc
`verify(mock, times(2)).metoda(arg)`	apelul a avut loc de două ori
`verifyNoInteractions(mock)`	mock-ul n-a fost atins deloc
`any()`, `anyInt()`, `anyString()`	orice argument de acel tip

@ExtendWith, @Mock, @InjectMocks

```
@ExtendWith(MockitoExtension.class)
class ProductCommandServiceImplTest {

    @Mock
    private ProductRepository productRepository;
    @Mock
    private ProductMapper productMapper;
    @Mock
    private Product product;

    @InjectMocks
    private ProductCommandServiceImpl service;
```

Adnotare	Ce face
`@ExtendWith(MockitoExtension.class)`	înaintea fiecărui test, Mockito completează câmpurile de mai jos, din nou
`@Mock`	un mock nou de tipul câmpului
`@InjectMocks`	creează clasa testată prin constructorul cel mai mare și îi dă mock-urile de același tip

`product` nu intră în constructor: îl dai tu ca argument în test (`service.addProduct(product)`).

Doar ce primești prin constructor poate fi înlocuit

```
public ProductCommandServiceImpl() {
    this(ProductRepositorySingleton.getINSTANCE(), new ProductMapper());
}

public ProductCommandServiceImpl(ProductRepository productRepository,
                                 ProductMapper productMapper) {
    this.productRepository = productRepository;
    this.productMapper = productMapper;
}
```

Aplicația folosește constructorul fără argumente, la fel ca înainte. Testul folosește constructorul cu argumente, prin `@InjectMocks`, și îi dă mock-urile. Dacă serviciul ar avea doar constructorul care cheamă `getINSTANCE()`, testul ar lucra cu repository-ul adevărat, care citește și scrie `Products.txt`.

ArgumentCaptor: obiectul creat în metodă

Când metoda testată face singură `new Product(...)` și îl dă repository-ului, n-ai un mock pe care să-l verifici. Îl prinzi la intrarea în mock:

```
ArgumentCaptor<Product> captor = ArgumentCaptor.forClass(Product.class);

service.createProduct(request);

verify(productRepository).addProduct(captor.capture());
assertEquals("Laptop", captor.getValue().getName());
```

`captor.capture()` stă în locul argumentului din `verify`; după, `getValue()` e obiectul primit.

Mockito e strict

Un `when` pe care testul nu-l folosește face testul să pice cu `UnnecessaryStubbingException`. Pui în fiecare test doar răspunsurile de care are nevoie. Așa, testul spune exact ce folosește metoda.

assert verifică rezultatul, verify verifică ce s-a cerut

```
@Test
void addProductThrowsWhenNameAlreadyExists() {
    // Arrange
    when(product.getName()).thenReturn("Laptop");
    when(productRepository.findProductByName("Laptop"))
            .thenReturn(Optional.of(otherProduct));

    // Act + Assert
    assertThrows(ProductAlreadyExistsException.class,
            () -> service.addProduct(product));
    verify(productRepository, never()).addProduct(any());
}
```

Dacă serviciul ar salva produsul **și apoi** ar arunca excepția, `assertThrows` tot ar trece; `verify(..., never())` prinde și cazul ăsta. Când metoda întoarce `void`, `verify` e singurul fel în care afli ce a făcut.